TradeX B.Tech Capstone — Technical Defense Guide

Person 2: Exchange Matching Engine & High-Frequency Order Architect

CANDIDATE / DEFENDER: Person 2 **PRESENTATION TIME:** 5 to 7 Minutes

ENGINEERING ROLE: Exchange Matching Engine & High-Frequency Order Architect

CORE DOMAIN: Continuous Double-Auction Matching Engine, Binary Heaps (Priority Queues), Order Books, Price-Time Priority FIFO Queues, Binary Search Tree (BST) Price Indexing, Order Pipeline

ASSIGNED REPOSITORY FILES:
- **backend/include/dsa/MatchingEngine.hpp**
- **backend/include/dsa/OrderBook.hpp**
- **backend/include/dsa/OrderQueue.hpp**
- **backend/include/dsa/BST.hpp**
- **backend/include/services/OrderService.hpp**

1. Executive Summary & Architectural Ownership

Person 2 owns the institutional core of TradeX: the Continuous Double-Auction matching engine and order execution pipeline. Every time a user places an order on the frontend, Person 2's code validates margin, organizes bids and asks in native C++ binary priority queues (Max-Heap for buyers, Min-Heap for sellers), resolves price-time priority using FIFO queues, matches orders deterministically at resting order prices, and exposes Level-2 5-depth Market Depth quotes. Person 2 also implements a Binary Search Tree (BST) for ascending price ladder indexing and traversals.

Core Defense Pitch: *"I implemented the core financial exchange matching engine in modern C++. My architecture maintains an active order book per equity instrument using two inverted binary heaps: a Max-Heap ensuring buyers offering the highest prices execute first, and a Min-Heap ensuring sellers offering the lowest prices execute first. Whenever Best Bid ≥ Best Ask, my Continuous Double-Auction matching engine executes trades at the maker price in O(log N) time, handling partial fills, timestamp tie-breaking via FIFO queues, and dynamic price indexing with a Binary Search Tree."*

2. Assigned Files & Detailed Code Responsibilities

backend/include/dsa/OrderBook.hpp (Dual-Heap Level-2 Order Book):
- Implements LimitOrder struct with id, userId, price, quantity, and timestamp.
- **Max-Heap for Bids:** Comparator BuyComparator: Highest price has top priority. If prices match, older timestamp executes first.
- **Min-Heap for Asks:** Comparator SellComparator: Lowest price has top priority. If prices match, older timestamp executes first.
- getSnapshot(int depth = 5): Aggregates order levels into top 5 price bands, volume sums, and order counts for frontend market depth.

backend/include/dsa/MatchingEngine.hpp (Continuous Double-Auction Core):
- matchOrders(OrderBook& book): Core algorithmic loop running while book.hasBuyOrders() && book.hasSellOrders().
- **Execution Condition:** Checks bestBuy->price >= bestSell->price. If true, matched quantity = min(buyQty, sellQty).
- **Execution Price Determination:** Follows the *Maker-Taker Principle*: execution price matches the resting order placed earlier in time.
- **Quantity Decrements & Heap Eviction:** Subtracts matched quantity; pops fully filled orders (popBuy() / popSell()); triggers TradeCallback.

backend/include/dsa/OrderQueue.hpp (Price-Time Priority Queue):
- Implements a FIFO queue (using std::deque<std::shared_ptr<LimitOrder>>) to guarantee fair execution order among traders

bidding identical prices.
• Supports O(1) push back, O(1) pop front, and O(N) linear order cancellation.

backend/include/dsa/BST.hpp (Binary Search Tree for Dynamic Price Indexing):
• Node structure: BSTNode storing price, symbol, volume, left, and right pointers.
• Insertion & Search: Average O(log N) binary tree insertion and lookup.
• Traversals: Implements **Inorder** (ascending sorted price ladder), **Preorder**, and **Postorder** tree walks.
• Extreme price finding: findMin() and findMax() in O(h) time by following leftmost/rightmost pointer chains.

backend/include/services/OrderService.hpp (Order Placement Pipeline):
• placeOrder(): Validates margin, generates order ID, and wraps execution in a MySQL database transaction.
• Handles Product Types: **CNC** (Cash & Carry Delivery) vs **MIS** (Intraday Margin).
• Dispatches matched trades to PortfolioService and WalletService to adjust holdings, cash balances, and ledger.
• cancelOrder(): Removes open resting limit orders from memory books and marks order as CANCELLED in MySQL.

3. Data Structures & Algorithms Implemented by Person 2

Data Structure / Algorithm	Time Complexity	Space Complexity	Engineering Role in TradeX
Binary Max-Heap (Bids)	Push: O(log N) Top: O(1) Pop: O(log N)	O(N) contiguous array	Guarantees the highest-paying buyer always has top priority for incoming market orders.
Binary Min-Heap (Asks)	Push: O(log N) Top: O(1) Pop: O(log N)	O(N) contiguous array	Guarantees the lowest-cost seller always has top priority for incoming buy orders.
Continuous Double-Auction Matching	O(M log N) where M = executed matches	O(1) Auxiliary	Greedy matching algorithm that clears orders when Bid ≥ Ask at the resting order's price.
FIFO Order Queue	Enqueue: O(1) Dequeue: O(1)	O(K) per price level	Enforces strict timestamp priority when multiple limit orders arrive at the exact same price.
Binary Search Tree (BST)	Avg: O(log N) Worst: O(N)	O(N) pointer nodes	Dynamic price ladder indexing and ascending Inorder traversals for visual market ladders.

4. How Person 2's Backend Connects to the Frontend

• **Order Placement Modal:** When a user clicks "BUY" or "SELL" on Markets.tsx, frontend/src/components/OrderModal.tsx submits an HTTP POST to /api/v1/orders. OrderService::placeOrder parses the request, pushes it into Person 2's dual-heap order book, and triggers MatchingEngine::matchOrders.
• **Market Depth (Order Book) Component:** frontend/src/components/OrderBook.tsx polls /api/v1/dsa/orderbook/:symbol every 2.5 seconds. OrderBook::getSnapshot(5) extracts top 5 bids and asks, displaying cumulative quantities and depth bars.
• **DSA Lab Tab 2 & Tab 3:** DsaLab.tsx visualizes the live binary heap priority queues and allows interactive BST price insertions via /api/v1/dsa/bst/insert.

5. B.Tech Defense & Viva Voce Q&A; (Ready to Speak)

Q1: Why are Priority Queues (Binary Heaps) superior to sorted arrays or linked lists for matching engines?
Answer: *"In a live exchange, orders arrive and cancel at thousands of ticks per second. A sorted array requires O(N) time for every insertion due to shifting elements in memory. A linked list allows O(1) insertion only if the pointer is already known, but finding the sorted position takes O(N). A Binary Heap offers optimal O(log N) insertion and deletion with O(1) peek time for the best bid or ask, stored in contiguous memory for maximum CPU L1/L2 cache locality."*

Q2: What is the Continuous Double-Auction mechanism and how is the execution price determined?
Answer: *"A Continuous Double-Auction continuously matches buyers and sellers as orders arrive. The crossing condition is Best Bid ≥ Best Ask. When a trade occurs, the execution price is determined by the resting order (the Maker order that was already sitting in the book), rather than the incoming aggressive Taker order. This eliminates front-running and provides price improvement to the taker."*

Q3: How do you handle partial fills when order quantities do not match?
Answer: *"In MatchingEngine::matchOrders, the executed trade quantity is calculated as min(buyOrder->quantity, sellOrder->quantity). We decrement this matched quantity from both orders. If an order's quantity reaches zero, it is popped from the heap. If remaining quantity > 0, it remains at the top of the heap to match against subsequent orders."*